Esterian Conquest

Sysop Manual

Version 1.0.0-beta.1 — Beta

Revision date: March 27, 2026

Contents

1. Introduction

Esterian Conquest (EC) is a multi-player strategy game originally written for DOS-era BBS systems. The Rust-native stack reimplements the game engine, player client, and admin tooling for modern systems while preserving classic compatibility at a well-defined import/export boundary.

This manual is for the **sysop** — the person responsible for hosting a game instance. It covers:

- installing and initializing a game
- configuration and theming
- recommended Nostr hosting with `ec-sysop nostr`
- localhost and direct deployment
- legacy BBS door compatibility
- player management
- turn processing and maintenance

For player-facing rules and gameplay, see the **Player Manual**.

This manual is the authoritative sysop manual for the Rust edition of Esterian Conquest. The preserved original `.DOC` set in `original/v1.5/` remains historical reference material and an ambiguity fallback for classic operator intent, not a higher-authority replacement for the current Rust manuals.

1.1. Terminology

game directory The directory containing `ecgame.db` and related files for a single running game instance. Passed to all tools via `--dir`.

`ec-sysop` The public Rust command-line sysop tool for campaign creation maintenance, and Nostr hosting.

`ec-connect` The beta-quality player-side connection client for the recommended hosted flow. It manages the player's Nostr identity, joins games by invite code, downloads the static starmap bundle on first join, and opens the SSH-backed `ec-game` session.

`ec-game` The Rust TUI player client.

`ecgame.db` The SQLite database that is the runtime source of truth for the Rust engine.

`themes/` Subdirectory containing theme KDL files. `themes/tokyo_night.kdl` is the default and is bootstrapped on first run alongside the other bundled themes. Sysops can add their own theme files here.

`config.kdl` The sysop-managed runtime configuration file in the game directory. Bootstrapped from the bundled default on first run. Controls snoop mode, session timeouts, inactivity thresholds, game name, and theme path. Changes take effect on the next `ec-game` startup.

2. Installation and Requirements

2.1. System Requirements

- Linux, macOS, or Windows
- Rust toolchain (stable) to build from source, or a pre-built binary release
- For the recommended public-hosted flow: an SSH-accessible host plus a Nostr relay reachable by players
- A terminal emulator for localhost or direct SSH play
- A BBS server with socket support only if you specifically want legacy door deployment

2.2. Building from Source

From the repository root:

```
cargo build --release -p ec-sysop -p ec-game -p ec-connect
```

The release binaries will be in `target/release/`.

2.3. Directory Layout

A running game occupies a single directory. A minimal populated game directory looks like:

```
/path/to/mygame/
  ecgame.db           runtime database (SQLite)
  config.kdl          sysop runtime config (bootstrapped on first run)
  themes/             theme directory (bootstrapped with tokyo_night plus bundled
alternates)
  exports/            default export root for classic .DAT output
  queue/              default turn order queue directory
```

All tools take `--dir /path/to/mygame` to locate the game.

3. Game Setup

3.1. Initializing a New Game

Create a new game with `ec-sysop new-game`:

```
ec-sysop new-game /path/to/mygame --players 4 --seed 1515
```

This creates a fresh campaign directory with `ecgame.db`, classic auxiliary files, `config.kdl`, and a `themes/` subdirectory containing the bootstrapped theme files shipped with `ec-game`.

`config.kdl` is the only sysop-edited KDL file in the public workflow. An internal `ec-cli` setup-preset format still exists for reproducible tests and harness work, but normal sysop operation does not use it.

The supported public creation flags are:

Flag	Type	Description
`--players`	integer	Number of empires. Supported range: 1–25. Defaults to `4`.
`--seed`	integer	Optional campaign seed for reproducible map generation.

4. Recommended Hosted Play

For new public campaigns, the recommended deployment path is `ec-sysop nostr` plus `ec-connect`. In that model, the sysop runs the normal Rust engine on a host, publishes the Nostr-facing daemon, and players join from their own machines with invite codes. The daemon handles identity and seat routing; the live game still runs inside `ec-game` over SSH. This keeps the original asynchronous EC rhythm without requiring per-player Unix account management or BBS door middleware.

A minimal hosted setup looks like:

```
ec-sysop new-game /path/to/mygame --players 4 --seed 1515
ec-sysop nostr init
ec-sysop nostr serve
```

After the daemon is running, give each player an invite code, the daemon's public key, and the relay URL they should use. That relay may be one you host yourself or a trusted public relay, but it should be treated as part of the normal hosted setup contract. The player-facing join command is:

```
ec-connect --join amber-river@play.example.com --gate npub1... --relay wss://relay.
example.com
```

On first join, `ec-connect` creates or unlocks the player's encrypted identity, binds that key to the seat, downloads the static starmap bundle, and opens the SSH-backed `ec-game` session. Returning players reconnect through `ec-connect` rather than launching `ec-game` directly. After a successful join, `ec-connect` caches the exact relay per game, so most later reconnects do not need the relay entered again.

5. Game Directory Structure

5.1. Core Files

`ecgame.db` The SQLite runtime database. All game state lives here. Do not edit manually; use `ec-sysop` for normal operator actions.

`themes/` Theme KDL files for `ec-game`. Bootstrapped on first run with `themes/tokyo_night.kdl` (the default) plus the other bundled alternates. Sysop-owned once created; not silently overwritten. See Section 7.

`config.kdl` Sysop runtime configuration. Bootstrapped from the bundled default on first run. Edit to change snoop mode, session timeouts, inactivity thresholds, game name, or theme path. See Section 6.

5.2. Subdirectories

`exports/` Default root for classic `.DAT` export output. Can be overridden with `--export-root` or the `EC_CLIENT_EXPORT_ROOT` environment variable.

`queue/` Default directory for turn order queue files. Can be overridden with `--queue-dir` or the `EC_CLIENT_QUEUE_DIR` environment variable.

6. Configuration

6.1. config.kdl

`config.kdl` is the sysop runtime configuration file. It lives in the game directory alongside `ecgame.db`. If absent when `ec-game` starts, it is bootstrapped from the bundled default automatically.

Changes to `config.kdl` take effect on the next `ec-game` startup. The runtime policy settings backed by `SETUP.DAT` bytes are applied into the runtime database at that point; no manual database edits are required.

6.1.1. Full Example

```
// Display name shown in the main menu header.
game_name "Esterian Conquest"

// Theme file (relative to game directory, or absolute path).
// Shipped themes live under themes/. Tokyo Night is the default.
// Omit to use themes/tokyo_night.kdl.
// theme "themes/tokyo_night.kdl"

// Sysop snoop: set to #false to disable.
snoop #true

// Session timeout and timing policies.
session {
    // Minutes of inactivity before timeout (0–120).
    max_idle_minutes 10
    // Minimum time granted per session in minutes (0–120).
    minimum_time_minutes 0
    // Apply timeout to local (non-remote) sessions.
    local_timeout #false
    // Apply timeout to remote sessions.
    remote_timeout #true
}

// Inactivity thresholds (in turns). Set to 0 to disable.
inactivity {
    // Purge a player after this many inactive turns (0–100).
    purge_after_turns 0
    // Put a player on autopilot after this many inactive turns (0–100).
    autopilot_after_turns 0
}

// Optional BBS/dropfile seat reservations by caller alias.
reservations {
    seat player=1 alias="SYSOP"
    seat player=2 alias="NightShade"
}
```

6.1.2. Top-Level Fields

Field	Type	Default	Description

`game_name`	string	`"Esterian Conquest"`	Display name shown in the main menu header.
`theme`	string	*(absent)*	Theme file path, relative to the game directory. Omit to use `themes/tokyo_night.kdl`. Example: `"themes/gruvbox.kdl"`.
`snoop`	bool	`#true`	Enable sysop snoop mode.
`reservations`	block	*(absent)*	Optional BBS/dropfile seat reservations by caller alias.

6.1.3. `session` Block

Field	Type	Default	Description
`max_idle_minutes`	integer	`10`	Minutes of inactivity before session timeout. Range: 0–120.
`minimum_time_minutes`	integer	`0`	Minimum session time guarantee in minutes. Range: 0–120.
`local_timeout`	bool	`#false`	Apply timeout to local (non-remote) sessions.
`remote_timeout`	bool	`#true`	Apply timeout to remote sessions.

6.1.4. `inactivity` Block

Field	Type	Default	Description
`purge_after_turns`	integer	`0`	Turns of inactivity before a player is purged. `0` = disabled. Range: 0–100.
`autopilot_after_turns`	integer	`0`	Turns of inactivity before autopilot engages. `0` = disabled. Range: 0–100.

6.1.5. `reservations` Block

Field	Type	Default	Description
`seat player=<N> alias="NAME"`	entry	*(absent)*	Reserve empire slot `N` for a BBS/dropfile caller alias. Alias matching is ASCII case-insensitive.

NOTE: `config.kdl` is for sysop use only. Players do not interact with it. Fields not present in the file use their default values. Omitting a field is equivalent to setting it to its default.

6.2. Environment Variables

Variable	Description
`EC_CLIENT_EXPORT_ROOT`	Override the default export root directory.
`EC_CLIENT_QUEUE_DIR`	Override the default turn queue directory.

COLORTERM	Color depth hint. truecolor or 24bit enables 24-bit color.
TERM	Terminal type. A value containing 256color enables 256-color mode.

7. Theming

ec-game uses a file-driven theme system. config.kdl defines the campaign's default theme, while local-terminal players can choose among the available themes from the client's C>olor Theme picker. Each player's last local theme choice is saved in ecgame.db as a per-player preference rather than by rewriting theme files. In BBS door mode, the client keeps the classic A>nsi color ON/OFF toggle and starts from the campaign default theme.

7.1. Theme File Location

ec-game resolves the theme in this order:

1. If <game_dir>/config.kdl contains a theme directive, use that path (relative to game_dir).
2. Otherwise, use <game_dir>/themes/tokyo_night.kdl.
3. If themes/tokyo_night.kdl does not exist, create the themes/ directory and bootstrap it from the bundled default.

config.kdl itself is bootstrapped on first run if absent, so it is always present by the time this resolution runs. The theme directive within it is optional; omitting it falls through to step 2.

On parse error, the bundled default is used so a corrupted theme never prevents players from connecting.

7.2. Player Theme Picker

From the Main Menu and First Time Menu in local-terminal sessions, players can open C>olor Theme to preview and apply the themes currently available in <game_dir>/themes/. The picker stays open after Enter so players can try several looks before returning to the menu with Q.

The picker exposes all file-backed themes currently present in themes/, including the bundled tokyo_night theme and the additional shipped alternates. It also exposes a synthetic Mono option, which applies a monochrome projection over the current theme for players who prefer a plain white-on-black display.

Joined players save their selected local theme immediately as a per-player preference in ecgame.db. A player choosing a theme from First Time Menu before fully joining uses it for that session, and the preference is saved when the join finishes successfully.

If a stored player theme key later points to a missing or invalid file, ec-game falls back to tokyo_night automatically. If a color theme still cannot be materialized, Mono remains the safe last-resort display.

7.3. Theme File Format

A theme file is a KDL document. Each visual element is declared as a style node with a name and child fg, bg, and optional bold fields.

```
style "body" {
    fg "white"
    bg "black"
}

style "logo" {
    fg "bright_blue"
    bg "black"
    bold #true
}

style "selected" {
    fg "black"
    bg "bright_blue"
}
```

The star decoration colors are declared separately:

```
star-colors "bright_blue" "bright_white" "white" "bright_yellow" "yellow" "bright_red"
```

7.4. Color Formats

Three color formats are supported:

Format	Example	Notes
Named ANSI-16	`"bright_blue"`	Safe for all terminals including BBS/door clients. Recommended default.
256-color index	`"idx:208"`	Requires `--color-mode 256` or truecolor. Downgraded to nearest named color in 16-color mode.
24-bit hex RGB	`"#ff8800"`	Requires `--color-mode truecolor`. Downgraded gracefully in lower color modes.

`themes/tokyo_night.kdl` is the default: a rich dark palette using 24-bit hex colors for modern SSH and local terminals, downgraded gracefully to 256-color or named ANSI-16 on terminals that do not support truecolor. `themes/mag16.kdl` is the ANSI-16 native alternative: a restrained dark palette using only named 16-color values, safe for all terminal types including BBS door clients. The shipped bundle also includes several other alternates. All hex colors degrade gracefully — they are automatically mapped to the nearest 256-color index or 16-color name when needed. Custom themes may use any of the three color formats.

7.5. Color Mode

`ec-game` selects a color mode at startup:

Mode	Description
`ansi16`	Classic 16-color ANSI. Safe for BBS/door and all terminal types.
`256`	256-color xterm palette.
`truecolor`	24-bit RGB. Best for modern local or SSH terminals.
`auto`	Detected from `COLORTERM` and `TERM` environment variables.

Override with the `--color-mode` flag:

```
ec-game --dir /path/to/mygame --player 1 --color-mode truecolor
```

When `--encoding cp437` is specified (BBS door mode), `ec-game` defaults to `ansi16` regardless of the environment, unless `--color-mode` is set explicitly.

7.6. Semantic Style Tokens

All required style tokens:

Token	Purpose
`body`	Default body text.
`title`	Screen and section titles.
`shell_title`	Outer shell border title text such as the `EC CONNECT` frame title. This should normally use the same background as `body` with a more prominent foreground accent.
`shell_label`	Outer shell border identity label text such as the active alias or shortened `npub` shown on the right side of the `EC CONNECT` frame. This should normally use the same background as `body` with its own accent color distinct from `shell_title`.
`menu`	Menu item text.
`menu_hotkey`	Menu hotkey letters.
`prompt`	Input prompt text.
`prompt_angle_delimiter`	Angle prompt punctuation such as the `< >` around explicit quit/cancel tokens like `<Q>`.
`prompt_square_delimiter`	Square prompt punctuation such as the `[ ]` around defaults like `[03,03]` or yes/no defaults like `[Y]`.
`prompt_hotkey`	Prompt hotkey letters.
`prompt_notice_action`	Action-notice accent in prompts.
`bright`	Emphasized body text.
`logo`	Title logo decoration.
`intro_accent`	Intro screen accent color.
`intro_tribute`	Intro screen tribute line.
`stardate_label`	Stardate label.
`stardate_week`	Stardate week value.
`stardate_year`	Stardate year value.
`error`	Error messages.
`notice`	Warning / notice messages.
`status_label`	Status bar labels.
`status_value`	Status bar values.
`table_chrome`	Table borders and separators.

`table_header`	Table column headers.
`table_body`	Table body rows.
`disabled_row`	Disabled / unavailable table rows.
`selected`	Selected / highlighted row.
`alert`	High-priority alert text.
`help_header`	Help overlay section headers.
`help_panel`	Help overlay body text.
`map_dot`	Star map background dots.
`map_crosshair`	Star map crosshair / cursor.
`map_center`	Star map center marker.
`quote`	Quote / flavor text body.
`quote_author`	Quote attribution.
`report_header`	Report page headers.
`indicator_on`	Active indicator (e.g. status lit).
`indicator_off`	Inactive indicator.

The prompt delimiters are styled separately from the key text they contain. In command rails and inline prompts, the inner letters still use `prompt_hotkey`, while `< >` and `[ ]` come from their own delimiter styles. This lets a theme keep the command keys legible while giving quit markers and defaults their own visual distinction.

7.7. Mono Theme

`Mono` is one of the local `C>olor Theme` picker entries. It applies a monochrome projection over the active theme and can be selected and saved like any other local player theme preference. In BBS door mode, monochrome output still comes from the separate `A>nsi color ON/OFF` toggle rather than the theme picker.

8. SSH Access

The recommended hosted path above already uses SSH under the hood: players enter through `ec-connect`, and the daemon opens a PTY running `ec-game`. You can also run `ec-game` over SSH directly when you want a private shared-host setup, manual debugging, or a simple trusted deployment without the Nostr invite flow.

`ec-game` runs cleanly over SSH. No special flags are required for modern terminal sessions.

Color mode is auto-detected from the environment:

- `COLORTERM=truecolor` → 24-bit RGB
- `TERM` containing `256color` → 256-color
- Otherwise → 16-color ANSI fallback

Force a specific mode with `--color-mode` if your SSH setup does not propagate `COLORTERM` reliably:

```
ec-game --dir /path/to/mygame --player 1 --color-mode 256
```

UTF-8 encoding (the default) is correct for SSH sessions on modern terminals. Use `--encoding cp437` only if you are proxying through a BBS or a CP437 terminal emulator over SSH.

9. Local and Direct Play

Localhost play remains a fully supported secondary mode for solo campaigns, hotseat sessions, and sysop testing. Run `ec-game` directly in your terminal:

```
ec-game --dir /path/to/mygame --player 1
```

Color mode and encoding default to `auto` / `utf8`, which is correct for modern terminal emulators. The client detects `COLORTERM` and `TERM` automatically.

10. Legacy BBS Door Setup

`ec-game` can still run as a BBS door. This path is preserved for classic-host compatibility, not as the primary recommendation for new public campaigns. In door mode, the client reads from and writes to a socket instead of the local TTY, using CP437 encoding and 16-color ANSI.

10.1. Flags for Door Mode

```
ec-game \
  --dir /path/to/mygame \
  --player <1-based index> \
  --encoding cp437 \
  --color-mode ansi16
```

`--encoding cp437` switches box-drawing characters and other extended characters to the CP437 code page, which is required for correct rendering in classic ANSI-aware BBS terminals (SyncTERM, NetRunner, etc.).

When `--encoding cp437` is active, `--color-mode` defaults to `ansi16` automatically. Override only if you know your BBS clients support a richer color depth.

10.2. Drop File

`ec-game` can read a BBS drop file directly with `--dropfile <path>`:

```
ec-game \
  --dir /path/to/mygame \
  --dropfile /path/to/DOOR32.SYS
```

Supported drop file formats (auto-detected by filename, case-insensitive):

- `DOOR32.SYS` — modern standard (Enigma, Mystic, Talisman, etc.)
- `DOOR.SYS` — legacy, widest BBS software support
- `CHAIN.TXT` — WWIV format

The drop file supplies the player alias and session time limit. Explicit CLI flags always override drop file values. When `--dropfile` is given and `--encoding` is not, encoding defaults to `cp437` automatically.

`--timeout <minutes>` sets a session time limit independently of a drop file.

Reserve seats in `config.kdl` when you want the caller alias to determine the empire automatically:

```
reservations {
  seat player=1 alias="SYSOP"
  seat player=2 alias="NightShade"
}
```

With that in place, a reserved caller can launch with `--dropfile` alone. If the caller alias is not reserved, `--player` is still required. If both `--player` and `--dropfile` are supplied for a reserved caller, they must agree on the same empire slot.

> **NOTE:** The original DOS `ECGAME.EXE` v1.5 expects a strict 32-line WWIV-style `CHAIN.TXT` drop file. Enigma BBS-generated `DOOR.SYS` / `DORINFO` files will crash `ECGAME.EXE`. See `docs/sysop/enigma-bbs-setup.md` for the full legacy DOS door path if you need to host the original binary.

10.3. Enigma BBS (Rust Client)

The native Rust `ec-game` door is now verified on both Mystic and ENiGMA½. For ENiGMA, use the `abracadabra` module with `dropFileType: DOOR32`, `io: stdio`, and `encoding: cp437`. Pass `--dir`, `--dropfile`, `--encoding cp437`, and `--color-mode ansi16` to the client, or use the helper wrapper at `tools/bbs/run_ec_rust.sh`. Use `--player` for unreserved callers, or reserve the alias in `config.kdl` and let `--dropfile` resolve the seat.

If Enigma writes a `DOOR32.SYS`, you can pass it directly:

```
ec-game \
  --dir /path/to/mygame \
  --dropfile /path/to/DOOR32.SYS
```

For a fuller ENiGMA½ Rust-door setup, including a ready `abracadabra` configuration, see `docs/sysop/enigma-rust-setup.md`.

In BBS door mode, the reliable control contract is:

- `HJKL` for movement
- `^U` / `^D` for paging
- `Q` or `Esc` for back/quit

Do not rely on arrows or `PgUp` / `PgDn` for normal play through BBS hosts.

11. File-Based Turn Submission

For localhost, shared-host, or custom-client workflows, players can submit orders by writing a KDL turn file and passing it to `ec-game submit-turn`. Use `--check` to validate without writing, then run without it to apply:

```
ec-game submit-turn --check --dir /path/to/mygame --player 1 --file /path/to/player1-turn.kdl
ec-game submit-turn --dir /path/to/mygame --player 1 --file /path/to/player1-turn.kdl
```

The `--player` value must match the `turn player=...` header in the file. If any order in the file is invalid, the entire submission is rejected and nothing is written. This is a direct apply command, not a queue or upload inbox.

A minimal turn file:

```
turn player=1 year=3000
tax rate=37
```

A fuller example:

```
turn player=1 year=3000

tax rate=37

planet record=16 {
  build points=4 kind="scout"
}

fleet record=1 {
  order speed=3 kind="scout_system" x=16 y=13
}

message to=2 subject="Border" body="Watching the north lane."
```

For the full node reference and schema, see `docs/sysop/turn-kdl.md`.

12. Turn Processing and Maintenance

Run yearly maintenance with:

```
ec-sysop maint /path/to/mygame [turns]
```

`ec-sysop maint` advances the campaign in `ecgame.db`. EC does not schedule maintenance by itself. In a real deployment, invoke `ec-sysop maint` from your host scheduler or BBS tooling:

- a `systemd` timer
- `cron`
- a BBS event runner
- or manual sysop operation

Do not treat KDL config as a scheduler. `config.kdl` owns runtime policy such as theming, snoop, and inactivity thresholds. Maintenance timing belongs to the host.

13. Player Management

Inactive-player policy is configured in `config.kdl` under the `inactivity` block. The two public thresholds are:

- `purge_after_turns`
- `autopilot_after_turns`

These values are runtime policy, not setup-time game creation input.

13.1. Reserving Seats

To reserve empire slots for specific BBS users, add a `reservations` block to `config.kdl`:

```
reservations {
    seat player=1 alias="SYSOP"
    seat player=2 alias="NightShade"
}
```

Each `seat` entry binds one 1-based empire slot to one caller alias. Alias matching is ASCII case-insensitive, so `NightShade`, `nightshade`, and `NIGHTSHADE` are treated as the same reservation.

With reservations in place, launch `ec-game` with `--dropfile` and let the caller alias choose the empire automatically:

```
ec-game --dir /path/to/mygame --dropfile /path/to/DOOR32.SYS
```

Important rules:

- if the caller alias is reserved, `--player` becomes optional
- if the caller alias is not reserved, `--player` is still required
- if both `--player` and `--dropfile` are supplied for a reserved caller, they must match
- if a reservation conflicts with an already-stored different player handle, `ec-game` will stop with a clear error so the sysop can reconcile `config.kdl` and the runtime state

Reserving a seat does not by itself join or pre-name the empire. It only routes the caller to the intended slot. The usual first-time join flow still claims an open empire, and that first successful join records the caller alias into the player record for later logins.

14. Classic Compatibility

The Rust-native public deployment path is `ec-sysop nostr`, `ec-connect`, and `ec-game`.

Classic `.DAT` import/export, oracle runs against the original binaries, and other preservation workflows still exist, but they belong to the internal `ec-cli` developer/compatibility surface rather than the normal sysop manual.

15. CLI Reference

15.1. ec-sysop

```
ec-sysop <subcommand> [options]
```

Subcommand	Purpose
`new-game`	Create a fresh campaign directory. Public flags: `--players` and `--seed`.
`nostr init`	Initialize the Nostr-hosting identity and config for the recommended public multiplayer path.
`nostr serve`	Run the Nostr-facing daemon that authenticates players and launches `ec-game` sessions.
`maint`	Run one or more maintenance turns against `ecgame.db`.

15.2. ec-game

```
ec-game --dir <game_dir> [--player <1-based index>] [options]
ec-game submit-turn [--check] --dir <game_dir> --player <record> --file <turn.kdl>
```

Interactive client flags:

Flag	Description
`--dir <path>`	Game directory containing `ecgame.db`. Required.
`--player <N>`	1-based empire index. Required unless a reserved dropfile alias resolves the seat from `config.kdl`.
`--encoding <utf8|cp437>`	Output encoding. Default: `utf8`. Use `cp437` for BBS/door mode.
`--color-mode <ansi16|256|truecolor|auto>`	Color depth. Default: `auto` (env-detected). CP437 mode defaults to `ansi16`.
`--dropfile <path>`	Parse a BBS drop file (DOOR32.SYS, DOOR.SYS, or CHAIN.TXT). Supplies alias and timeout, defaults encoding to `cp437`, and can resolve the player seat through `config.kdl` reservations. Explicit flags always override except that `--player` must match a reserved alias when both are present.
`--timeout <minutes>`	Session time limit in minutes. Overrides any drop file value.
`--export-root <path>`	Override export directory. Default: `<game_dir>/exports`.
`--queue-dir <path>`	Override turn queue directory. Default: `<game_dir>/queue`.

`submit-turn` flags:

Flag	Description
`--check`	Validate the KDL file without mutating the campaign.
`--dir <path>`	Game directory containing `ecgame.db`. Required.
`--player <N>`	1-based empire index. Required, and must match the KDL header.
`--file <path>`	Turn submission KDL file to validate or apply. Required.

NOTE: `ec-game submit-turn` is all-or-nothing. If any command in the file is invalid, the entire submission is rejected and nothing is written to `ecgame.db`.

16. Theme Token Reference

The ANSI-compatible reference palette below matches the bundled `mag16` theme. The default bootstrapped game theme is still `tokyo_night.kdl`, but `mag16` is a useful baseline because it shows the semantic token split using portable named ANSI colors.

Token	fg	bg	bold
body	white	black	—
title	bright_blue	black	yes
menu	white	black	—
menu_hotkey	yellow	black	yes
prompt	white	black	—
prompt_angle_delimiter	green	black	yes
prompt_square_delimiter	red	black	yes
prompt_hotkey	yellow	black	yes
prompt_notice_action	bright_cyan	black	yes
bright	bright_white	black	yes
logo	bright_blue	black	yes
intro_accent	bright_blue	black	—
intro_tribute	bright_magenta	black	—
stardate_label	cyan	black	—
stardate_week	bright_cyan	black	—
stardate_year	yellow	black	—
error	red	black	yes
notice	bright_red	black	yes
status_label	white	black	—
status_value	white	black	—
table_chrome	blue	black	—
table_header	cyan	black	yes
table_body	bright_white	black	—
disabled_row	bright_black	black	—
selected	black	bright_blue	—
alert	bright_white	red	yes
help_header	bright_blue	black	yes
help_panel	white	black	—
map_dot	green	black	—
map_crosshair	bright_red	black	yes
map_center	bright_white	black	yes

quote	white	black	—
quote_author	white	black	—
report_header	cyan	black	—
indicator_on	bright_green	black	yes
indicator_off	bright_black	black	—

Star decoration colors (6 slots, cycling):

```
star-colors "bright_blue" "bright_white" "white" "bright_yellow" "yellow" "bright_red"
```